Statement	Valid	Invalid
`x = x + 1`		
`INPUT z`		
`IF y < 6 THEN`		
`add 3 to the value entered`		
`store the value entered in the variable newValue`		
`value1 = INPUT("Enter a number")`		
`IF number is more than 10`		
`WHILE number1 <> 0`		

10 Write a pseudocode algorithm to:
- take three values as input
- multiply the numbers together
- output the result.

11 Write a pseudocode algorithm to:
- take 10 numbers as input (using a loop)
- add together the values
- output the total with an appropriate message.

7.5 Coding

Once you have decomposed your problem into subproblems, and you have designed the algorithms using flowcharts and/or pseudocode then you can start writing the program in your chosen programming language. This is often referred to as **coding**.

This will also include some iterative testing. This is testing that is carried out while the program is being developed, for example, if you write the program code for one of the subproblems, then you will test it with a range of different data to make sure it is fully working before moving onto the next step.

7.6 Testing

When you have finished your program, you need to carry out **testing** to make sure it:

- fully works
- does not crash
- meets all requirements.

You will need to identify appropriate **test data** to use to test the program.
There are four types of test data:

- **Normal** – data that the program should accept.
- **Abnormal** – data that the program should not accept.
- **Extreme** – data that is at the edge of what is allowed.
- **Boundary** – data that is on the edge of being accepted and being rejected.

KEY WORDS

coding: the writing of a program using one or more programming languages.

testing: repeated use of a system to try all different possibilities to make sure the system is fully working and cannot be broken.

test data: the input data that is used to test a system.

normal test data: data that a program should accept.

abnormal test data: data that a program should not accept.

extreme test data: data that is on the edge of what is allowed.

boundary test data: data that is on the edge of being accepted, and data that is on the edge of not being accepted.

Example 1

A program allows the user to enter their age as a whole number between 10 and 100 years.

Test data type	Example data
Normal	Any number between 10 and 100 inclusive, e.g. 10, 20, 50, 100, etc.
Abnormal	Any data that is not a number between 10 and 100, e.g. 9, −1, 101, 200, "age", "30", 12.2.
Extreme	The largest number accepted – 100. The smallest number accepted – 10.
Boundary	Either side of the largest bound – 100 and 101. Either side of the smallest bound – 9 and 10.

Example 2

A user needs to enter a password at least 8 characters long.

Test data type	Example data
Normal	Any set of 8 or more characters, but they must be text (have speech marks), e.g. "abcdefgh" "12fghj567812".
Abnormal	Any set of characters with 7 or less characters, e.g. "1234567" , "seven".
Extreme	Any with 8 characters only, e.g. "12345678", "ajrkdhJK".
Boundary	Either side of the 8 character bound, 7 characters or 8 characters, e.g. "1234567", "12345678", "abcdefg", "iJHFD7hJ",

ACTIVITY 7.4

In pairs find a computer game to play. Anything as simple as a puzzle game, to a full 3D adventure game. Discuss how this game might have been tested. What data would be used as inputs?

Questions

12 Identify **three** different types of test data.

13 Why can you have the same test data as normal and extreme types of test data?

14 What is the difference between extreme and boundary test data?

15 A program asks the user to input a number between 1 and 10 (inclusive).

Identify test data for each of the test data types in the table:

Test type	Test data
Normal	
Abnormal	
Extreme	
Boundary	

7.7 Common algorithms

A **search algorithm** checks a set of data to identify whether a specific value exists in the data, or not. One example of this is a linear search.

A **sorting algorithm** takes a set of data and rearranges it to be in a specific order, e.g. in ascending alphabetical order. One example of this is a bubble sort.

KEY WORDS

search algorithm: a series of steps that searches for a specific value in a set of data.

sorting algorithm: a series of steps that will rearrange a set of data into an order, e.g. ascending numerical data.

COMPUTER SCIENCE IN CONTEXT

Sorting data and searching for data are activities that a very large proportion of programs will use at some point. For example, outputting a list of data may need sorting into order. A user may enter a username and the program needs to search for this in the set of valid usernames.

There are many different searching and sorting algorithms, each one has its benefits and drawbacks depending on the problem. This could be the number of data items it has to sort through, or whether the data is already in order to search for a value. It is up to the programmer to identify the algorithm that is most appropriate for their program to make sure it is as efficient as possible (both in the time it takes to run, and the amount of memory it needs to perform this task).

Linear search

In a program you might need to look for a specific value in a set of data. A **linear search** will check each item one at a time, starting with the first item and continuing until it either finds the item, or it checks the last value.

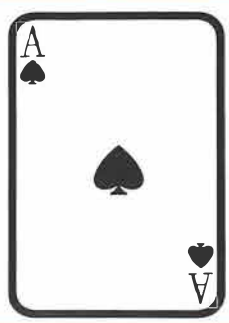
KEY WORD

linear search: a search algorithm that visits each item of data in turn to check whether it is the data being looked for.

ACTIVITY 7.5

Get a set of playing cards. Select 10 random cards and place them face down on the table in a row. Perform a linear search to see if the Ace of Spades is in the set.

1. Turn over the first card.
2. Is it the Ace of Spades?
3. If it is, then stop searching, you've found it.
4. If it isn't, turn over the next card. Repeat from 2.



CONTINUED

2 Recap how to calculate an average of a set of numbers.

3 30 numbers will need entering. Recap how to loop 30 times.

Practice

1 Write an algorithm to allow the teacher to enter the marks for all 30 students.

2 Amend the program to calculate and output the smallest (minimum) mark.

3 Amend the program to calculate and output the largest (maximum) mark.

4 Amend the program to calculate and output the average mark.

Challenge

Extend the program to count how many got below the average, and how many got above the average number of marks.

Extend the program to work out how many students got each level. 80 or more = distinction. 60 or more = merit. 40 or more = pass. Below 40 = fail.

Questions

21 Explain the difference between totalling and counting.

22 A program allows numbers to be entered between 20 and 200, and finds the smallest and largest entered. Identify appropriate values to initialise the variables `Smallest` and `Largest`.

23 Write an algorithm to input 40 numbers and output the total and average.

24 Write an algorithm to input 100 numbers and output the smallest and largest.

Validation on input

Validation is the checking of data that is input to make sure it is reasonable, and/or within set bounds. For example, making sure that a number is entered for an age or limiting the range of numbers that can be entered.

KEY WORD

validation: the checking of data to make sure it is reasonable and within set bounds.

ACTIVITY 7.7

Have you ever entered data to be told that it is invalid? What were you trying to enter, and why was it incorrect? Visit some websites that need you to enter some data (but stay safe and make sure it is not personal data).
What restrictions do they have? Are there some spaces that you need to enter data in? Do some of them give you a set structure to fill in, for example, a date as _ _ / _ _ / _ _ _ _?
Are these all controlled with validation rules?

There are different types of validation.

Range check

A **range check** assesses whether data is within one or two bounds. For example, an age must be between 0 and 100. A date must be before today's date.

Range checks can be programmed using selection to produce an error, or within a loop that keeps asking you enter a value until it is valid.

KEY WORD

range check: a type of validation that makes sure data is between the minimum and maximum.

Example 1

Using selection:

This algorithm will check whether a number is higher than 1 and less than 10.

```
Number ← INPUT("Enter a number between 1 and 10")
IF Number < 1 OR Number > 10 THEN
    OUTPUT("Invalid")
ELSE
    OUTPUT("Valid")
ENDIF
```

Example 2

Using a loop:

This algorithm will check if a number is higher than 1 and less than 10. It will continually ask for a value to be input until the number is valid.

```
Number ← INPUT("Enter a number between 1 and 10")
WHILE Number < 1 OR Number > 10 DO
    OUTPUT("Invalid please try again")
    Number ← INPUT()
ENDWHILE
```

Length check

A **length check** will check the number of characters that are present. This could in a string, for example, the length of "hello world" is 11, the space is also a character. It could be in a variable, for example, in this example, 8 will be output:

KEY WORD

length check: a type of validation that checks the number of characters is within a set limit.

```
TheData ← "123 ABC!"
OUTPUT(LENGTH(TheData))
```

The length of a piece of data can be found using a variety of commands depending on the language you are using. The following are all valid and there are many more:

```
theData.length()
LENGTH(theData)
LEN(theData)
theData.len
```

A length check can be programmed using selection or iteration.

Example 1

Using selection:

This algorithm will check if the data input has 10 or less characters to be invalid, otherwise (10 or more) it is valid.

```
Data ← INPUT()
IF LENGTH(Data) < 10 THEN
    OUTPUT("Invalid")
ELSE
    OUTPUT("Valid")
ENDIF
```

Example 2

Using iteration:

This algorithm will also check the length but will continually ask for this to be input until it is 10 or more characters long.

```
Data ← Input()
WHILE LENGTH(Data) < 10 DO
    OUTPUT("Invalid, please try again")
    Data ← INPUT()
ENDWHILE
```

Type check

Data can be in different forms, including an integer (whole number), real (decimal number), string (any characters), Boolean (true or false). There are two ways of writing a **type check**:

1. Use a function such as `.GetDataType()` to return the data type, that you can then compare to the one you want.
2. Use a function such as `.IsInteger()` to return True if it is an integer and False otherwise.

KEY WORD

type check: a type of validation that checks data is the correct data type.

Example 1

Using selection:

This algorithm will check if the data entered is an integer value to be valid.

```
INPUT Data
//check if the data entered is an Integer
IF Data.GetDataType() <> Integer THEN //if it is not an integer
    OUTPUT ("Invalid")
ENDIF
```

Example 2

Using iteration:

This algorithm will continually ask for the data to be input until it is a string value.

```
INPUT Data
//loop while the data entered is not a string
WHILE Data.IsString = FALSE DO
    OUTPUT("Invalid please try again")
    INPUT Data
ENDWHILE
```

Presence check

A **presence check** makes sure that some data has been entered. Programming languages can make use of the value `null`, or for a string value an empty string represented by `""`.

KEY WORD

presence check: a type of validation that makes sure data has been entered.

Example 1

Using selection:

This algorithm outputs Invalid if there is no data entered.

```
INPUT Data
IF Data = NULL THEN  //check if the data entered is null
   OUTPUT("Invalid")
ENDIF
```

Example 2

Using iteration:

This algorithm continually takes an input while there is no data entered.

```
INPUT Data
WHILE Data = "" DO //loop while there is nothing in data
   OUTPUT("Invalid please try again")
   INPUT Data
ENDWHILE
```

Format check

Some data may need to be entered in a specific way, for example, a date must be: _ _ / _ _ / _ _ _ _, where each space is a number – a **format check** makes sure the data is entered this way. An ID number may need to be 1 number followed by 3 characters.

KEY WORD

format check: a type of validation that makes sure the data meets a specific order, e.g. 1 number, then 3 letters.

Example 1

Using selection:

An ID number entered needs to be 1 number followed by 2 characters. This algorithm checks the characters and outputs whether it meets the required format.

```
INPUT IdNumber
//check if the first character is a number, and characters 2
//and 3 are strings
IF(SUBSTRING(IdNumber, 0, 1).IsNumeric = TRUE AND
   SUBSTRING(IsNumber, 1, 2) = TRUE) THEN
   OUTPUT("Valid")
ELSE
   OUTPUT("Invalid")
ENDIF
```

Note that `SUBSTRING(String, X, Y)` starts at character x in the string, and returns y number of characters.

Example 2

Using a loop:

This algorithm checks whether an input is in the format two numbers, /, two numbers, /, four numbers.

```
INPUT Date
//loop while the data is not in the correct format
WHILE (SUBSTRING(Date, 0, 2).IsNumeric = FALSE OR //2 numbers
       SUBSTRING(Date, 2, 1) <> "/" OR // 1st slash
       SUBSTRING(Date, 3, 2).IsNumeric = FALSE OR //2 numbers
       SUBSTRING(Date, 5, 1) <> "/" OR //2nd slash
       SUBSTRING(Date, 6, 4).IsNumeric = FALSE) DO // 4 numbers, year
   OUTPUT("Invalid")
   INPUT Date
ENDWHILE
OUTPUT("Valid")
```

Check digit

A **check digit** is calculated from a set of numbers, and is input with the numbers. When the data has been input, the calculation is performed on the data again and the result is compared with the final number (the check digit).

KEY WORD

check digit: a type of error detection method that is used for data entry. A calculation is performed on the data entered to create a value. Check digit values are compared to see if the data entered is correct.

Example

This algorithm calculates the check digit using the method 5 digit number input. 9999, 4 digits used to calculate 5th check digit, and compares it to the input value.

```
INPUT Code
//extract 1st digit
Digit1 ← Code DIV 1000
Code ← Code - (1000 * Digit1)
//extract 2nd digit
digit2 ← Code DIV 100
Code ← Code - (100 * Digit1)
//extract 3rd digit
Digit3 ← Code DIV 10
Code ← Code - (10 * Digit1)
//extract 4th digit
Digit4 ← DIV(Code, 1)
CheckDigit ← Code
//calculate check digit from data entered
//multiply by position and add together
Total ← (Digit1 * 4) + (Digit2 * 3) + (Digit2 * 2) + Digit3
NewCheckDigit ← MOD(Total, 11) //find Mod 11 of total
NewCheckDigit ← 11 - NewCheckDigit //subtract result from newCheckDigit
//check if the calculated check digit is the same as the last digit in the //code
IF NewCheckDigit = CheckDigit THEN
   OUTPUT("Correct data entry")
ELSE
   OUTPUT("Incorrect data entry")
ENDIF
```

You can find out more about check digits in Chapter 2.

Verification of data

When entering data from another source, for example, a paper copy, it is important to make sure you have entered exactly what was written originally. This is called **verification**, a check that you have copied the data accurately.

There is a difference between entering data accurately, and entering the correct data. Verification only checks you have copied it accurately, the actual data may still be incorrect if the original was incorrect.

Two forms of verification are a visual check and double entry.

A **visual check** is where you compare the data entered to the original. For example, reading each line from a paper copy and checking it is identical on the computer.

A **double entry check** is where the same data is entered twice, usually by different people. The computer will then check whether there are any differences in what they each entered.

KEY WORDS

verification: checking that data is entered accurately, that it is the same as the original.

visual check: comparing the data entered with the original side-by-side.

double entry check: two different people enter the same data which are then compared.

COMPUTER SCIENCE IN CONTEXT

Have you ever had to enter data into a program twice? You have most likely done this when entering your email address, or a password. Think about why it was important that your email and password were entered accurately. What could have happened if they were not?

PROGRAMMING TASK 7.6

A user needs to enter a series of data that each require validation. The table shows the data and the requirements:

Data	Validation requirements
Username	Minimum length of 5 characters.
Date of birth	In the format NN/NN/NNNN, e.g. 01/01/2020.
Type of character	Limited to: "Elf", "Fairy", "Gnome", "Magician".
Starting strength	A number between 1 and 5 inclusive.
Starting health	10 minus the starting strength input.

Getting started

For each of the validation requirements, decide on which type of validation you are going to use. Try and choose a different one for each value.

Recap the algorithms for each type, write an example beside each of the ones you have chosen.

CONTINUED

Practice

1 Write an algorithm to take as input the five values.

2 Amend the program to include the validation for each item of data.

3 Amend your program to output, for each data item, whether their data is valid or invalid.

Challenge

Extend the program to repeatedly ask for each item of data until they enter one that is valid. Then the next piece of data should be input.

Could any of the items of data have more than one validation rule used? Find at least one piece of data that can have two types of validation and implement it in your program.

ACTIVITY 7.8

Work in pairs to identify how many times you have come across validation or verification in a program or website. Compare your answers with another pair. How common are validation and verification? Are there any methods not identified here, for example, what is two-step verification?

Questions

25 What is validation?

26 Give **three** examples of validation checks.

27 What is the purpose of verification?

28 Identify which validation rule this algorithm uses:

```
INPUT Num
IF Num > 0 AND Num < 100 THEN
    OUTPUT "Valid"
ELSE
    OUTPUT "Invalid"
ENDIF
```

29 Identify which validation rule this algorithm uses:

```
REPEAT
    INPUT Value
UNTIL Value <> NULL AND Value <> ""
```

30 Write a validation routine to input a word and check that it is more than 10 characters long.

31 Write a validation routine to input a number and check that it is an integer value.

7.8 Trace tables

A **trace table** is a structure to help you follow an algorithm, for example, to find an error, or to work out what the algorithm does.

Each statement in an algorithm is run manually, and the values that are written to variables are written in the table in the order the changes are made.

Each column in the table is designated to one variable. There can also be a column for any outputs to be written in. User prompts are also entered in the trace table, these can be entered in a separate column or as an output (because they will be output). For example, x = input("Enter a number"), the text "Enter a number" is a user prompt and will need to be added to the trace table.

Specific inputs can be given to test different parts of the algorithm.

KEY WORD

trace table: a structure to complete when walking through an algorithm manually, where the values that change are written in each row.

SKILLS FOCUS 7.3

TRACE TABLE WITH SELECTION

To create a trace table you need to identify all of the variables in an algorithm. Each variable should have its own column in the trace table. There should also be an output column if the program will output any data or strings.

For example, trace the following algorithm with the input values 1 and 3.

```
INPUT Number1
INPUT Number2
IF Number1 > Number2 THEN
    OUTPUT(Number1)
ELSE
    OUTPUT(Number2)
ENDIF
```

There are **two** variables and there is an output (each variable has a column, and the output has a column).

number1	number2	OUTPUT

You then complete this table by reading through the algorithm one line at a time. It is important not to jump ahead and guess what the algorithm will do. Read one line and perform that action.

Run line 1 of the code (input values 1, 3):

```
INPUT Number1
INPUT Number2
IF Number1 > Number2 THEN
    OUTPUT(Number1)
ELSE
    OUTPUT(Number2)
ENDIF
```

CONTINUED

number1	number2	OUTPUT
1		

The first line inputs the first value given, 1.

Run line 2 of the code (input values 1, 3):

```
INPUT Number1
INPUT Number2
IF Number1 > Number2 THEN
    OUTPUT(Number1)
ELSE
    OUTPUT(Number2)
ENDIF
```

number1	number2	OUTPUT
1	3	

The second line inputs the second value given, 3.

Run the IF statement:

```
INPUT Number1
INPUT Number2
IF Number1 > Number2 THEN
    OUTPUT(Number1)
ELSE
    OUTPUT(Number2)
ENDIF
```

Use the values in the table. number1 is 1, number2 is 3.

IF 1 > 3 This is FALSE so run the ELSE

OUTPUT the value in number2

number1	number2	OUTPUT
1	3	3

ACTIVITY 7.9

Run the algorithm in Skills Focus 7.3 with the following input data:

a 10, 2

b 9, 9

c 20, 30

Peer Assessment

Compare your answers from running the algorithm with the different values with a partner. Did you get the same result? If not, work out which is correct by running the algorithms again.

SKILLS FOCUS 7.4

TRACE TABLE WITH LOOPS

When you are following an algorithm that includes a loop (iteration), some of the code will be repeated. To follow the code run one line at a time as before, and when you get to the end of the loop, go back to the start of the loop.

For example: trace the following algorithm with the input values 4, 3, 2, 1, 0.

```
01 Count ← 0
02 InputValue ← 1
03 Total ← 0
04 WHILE InputValue > 0 DO
05     INPUT InputValue
06     Total ← Total + InputValue
07     Count ← Count + 1
08 ENDWHILE
09 OUTPUT "The total is ", Total
10 OUTPUT "There were ", Count, " Numbers"
```

Start by creating the trace table with each of the variables and the output.
There are 3 variables in this algorithm:

Count	InputValue	Total	OUTPUT

1 Run the first 3 lines of code. These are the initialisations, lines 01, 02 and 03.

```
01 Count ← 0
02 InputValue ← 1
03 Total ← 0
```

Count	InputValue	Total	OUTPUT
0	1	0	

2 Run line 04.

```
04 WHILE InputValue > 0 DO
```

LOOP is 1 so this is TRUE meaning that the code inside the loop is run

Run line 05 with the first input value of 4.

```
05     INPUT InputValue
```

Count	InputValue	Total	OUTPUT
0	1	0	
	4		

When a variable has a new value, it is written below the previous one.

CONTINUED

3 Run lines 06 and 07.

```
06      Total ← Total + InputValue        Total ← 0 + 4
07      Count ← Count + 1                 Count ← 0 + 1
```

count	inputValue	total	OUTPUT
0	1	0	
1	4	4	

4 Line 08:

```
08  ENDWHILE
```

sends you back to line 04.

```
04  WHILE InputValue > 0 DO
```

`InputValue` is 4, so while 4 > 0 is TRUE to run line 05 with the second input value of 3.

```
05      INPUT InputValue
```

count	inputValue	total	OUTPUT
0	1	0	
1	4	4	
	3		

5 Lines 06 and 07 run.

```
06      Total ← Total + InputValue    Total ← 4 + 3
07      Count ← Count + 1             Count ← 1 + 1
```

count	inputValue	total	OUTPUT
0	1	0	
1	4	4	
2	3	7	

6 Line `08  ENDWHILE` returns to line 04.

```
04  WHILE InputValue > 0 DO     WHILE 3 > 0 is TRUE. Run lines 05, 06 and 07.
05      INPUT InputValue                    InputValue ← 2
06      Total ← Total + InputValue          Total ← 7 + 2
07      Count ← Count + 1                   Count ← 2 + 1
```

count	inputValue	total	OUTPUT
0	1	0	
1	4	4	
2	3	7	
3	2	9	

7 Line 08 again returns to line 04. WHILE 2 > 0 is TRUE. Run lines 05, 06 and 07.

```
05      INPUT InputValue                    InputValue ← 1
06      Total ← Total + InputValue          Total ← 9 + 1
07      Count ← Count + 1                   Count ← 3 + 1
```

CONTINUED

count	inputValue	total	OUTPUT
0	1	0	
1	4	4	
2	3	7	
3	2	9	
4	1	10	

8 Line 08 returns to line 04 again. WHILE 1 > 0 is TRUE. Run lines 05, 06 and 07.

```
05      INPUT InputValue                    InputValue ← 0
06      Total ← Total + InputValue          Total ← 10 + 1
07      Count ← Count + 1                   Count ← 4 + 1
```

count	inputValue	total	OUTPUT
0	1	0	
1	4	4	
2	3	7	
3	2	9	
4	1	10	
5	0	10	

9 Line 08 returns to line 04 again. WHILE 0 > 0 is FALSE this time, so the loop does not run.

Jump to line 09 run line 09 and 10.

```
09 OUTPUT "The total is ", Total
10 OUTPUT "There were ", Count, " numbers"
```

count	inputValue	total	OUTPUT
0	1	0	
1	4	4	
2	3	7	
3	2	9	
4	1	10	
5	0	10	The total is 10
			There were 5 numbers

Be careful with your outputs. They must match the output in the code. For example, '*The total is 10*' is correct, but '*total = 10*' is incorrect.

Questions

1. What do you record in a trace table?
2. When do you change the value in a column?

ACTIVITY 7.10

Trace the algorithm in Skills Focus 7.4 with the following inputs:

a 1, 6, 8, 2, 0

b 1, −1, 2

c 10, 20, 30, −5

Self Assessment

Trace the algorithms a second time and see whether you got the same result. If not, where did you go wrong?

7.9 Finding the purpose of an algorithm

A trace table can help you work out the purpose of an algorithm. By following an algorithm step-by-step you can see what happens to the values, what changes are made, and when these are made. From this you can determine the purpose of the algorithm.

SKILLS FOCUS 7.5

FINDING THE PURPOSE WITH A TRACE TABLE

Describe the processes in this algorithm and state its purpose.

```
INPUT Value1
INPUT Value2
INPUT Value3
IF Value1 > Value2 AND Value1 > Value3 THEN
   OUTPUT(Value1)
ELSEIF Value2 > Value1 AND Value2 > Value3 THEN
   OUTPUT(Value2)
ELSE
   OUTPUT(Value3)
ENDIF
OUTPUT(Value1 + Value2 + Value3)
```

1 First test the algorithm with a set of data that you come up with (if they are not provided for you). For example, test this algorithm using 1, 2 then 3.

2 Create a trace table by following the algorithm with the data input.

Value1	Value2	Value3	OUTPUT
1	2	3	3
			6

3 Repeat the process with a different set of data, for example, this time using 10, 5, 1.

4 Complete the trace table.

CONTINUED

Value1	Value2	Value3	OUTPUT
10	5	1	10
			16

Keep on repeating this process until you can identify the pattern.

In this case it is outputting the largest number input and then outputting the total of the 3 numbers input.

Questions

1 Two sets of data were used to run this algorithm. If you had to choose a third set what would you choose?

2 How do you know when to stop testing the algorithm with different data?

3 Describe the processes in this algorithm and state its purpose using a trace table.

```
Quantity ← 0
Total ← 0
Value ← 0
Small ← 999
WHILE Quantity < 5 DO
    INPUT Value
    IF Value >= 10 AND Value <= 100 THEN
        Quantity ← Quantity + 1
        Total ← Total + Value
        IF Value < Small THEN
            Small ← Value
        ENDIF
    ENDIF
ENDWHILE
OUTPUT("small = ", Small)
OUTPUT(Total / Quantity)
```

quantity	Total	value	small	OUTPUT

TIP

Copy the trace table and test the algorithm in Q3 with some sample data, e.g. 1, 100, 50, 10, 33, 3, 42.

SKILLS FOCUS 7.6

DESCRIBING AN ALGORITHM

When you are describing an algorithm you need to do more than repeat the line of code using English statements. For example, in the following code, for line 02 don't just say, 'it is a for loop from y is 0 to x.' This is stating what each part of the code is, but it does not describe the algorithm.

```
01 INPUT X
02 FOR Y ← 0 TO X
03     OUTPUT Y
04 NEXT Y
```

You can do a trace table to help you work out the purpose of the algorithm before describing it if that helps you. This algorithm has three parts: line 01, line 02 and line 03. These are summarised below:

- Takes a number as input. (line 01)
- Loops from 0 to the number that has been input. (line 02)
- Outputs the values from 0 to the number. (line 03)

From this you can develop a description of the algorithm, for example:

The algorithm takes a number from the user, and then outputs all of the numbers from 0 to the number input.

Questions

1 What methods can you use to work out the purpose of an algorithm?

2 Why should your description explain more than what each part of the code does?

3 Describe the purpose of this algorithm:

```
01 X ← -1
02 Count ← 0
03 WHILE Count < 10 DO
04     INPUT Y
05     IF Y > X THEN
06         X = Y
07     ENDIF
08     Count ← Count + 1
09 ENDWHILE
10 OUTPUT(X)
```

7.10 Finding errors in a program and correcting the errors

To find an error in an algorithm you need to know the purpose of the algorithm. You can then test the algorithm with different data, completing trace tables if needed, to find out what the algorithm actually does. Once you know the difference between what it does and should do, you can start to identify the errors and the changes you need to make.

COMPUTER SCIENCE IN CONTEXT

Computer programmers will come across errors all the time. These could be ones that they have made, or that they are testing or fixing for other people. There are lots of ways that they can find the errors, but many of them will use the two methods used here; using a trace table, and dry running by reading the code. There are other options often available to them as well, for example, features within the Integrated Development Environment (IDE) that they use to write the program. These features allow the programmer to see the values of variables and the outputs at each stage, without having to trace them manually.

SKILLS FOCUS 7.7

USING A TRACE TABLE TO FIND ERRORS IN A PROGRAM

Using a trace table will allow you to follow each step in the algorithm and by comparing this to the intended outcome you can find where it goes wrong, and then work out how to correct it.

For example, this program should take 5 numbers as input and output the total of all the numbers.

```
01 Total ← 1
02 Quantity ← 1
03 WHILE Quantity < 5 DO
04     INPUT Number
05     Total ← Number
06     Quantity ← Quantity + 1
07 ENDWHILE
08 OUTPUT(Total)
```

First dry run the algorithm. If not provided, you will need to identify some test data to use.

This algorithm should take 5 numbers as input, so you could use 1, 2, 3, 4, 5.

Then work out what the result of the algorithm should be. It outputs the total, so it should output: $1+2+3+4+5=15$

CONTINUED

Now draw a trace table for the algorithm (revisit the section on trace tables if needed) and then run each step of the algorithm, writing the values in the table.

Total	Quantity	Number	OUTPUT
1	1		
1	2	1	
2	3	2	
3	4	3	
4	5	4	

Now look at the results and where it has gone wrong:

1. The total is overwritten each time, it should be added to.
2. Total starts with 1 but this will add another 1 to the total, so it should be 0.
3. Only four numbers can be input, it should be five.

Let's take each of the problems one at a time to solve them.

1. The total should not be overwritten. Line 05 overwrites the total. This should be `Total ← Total + Number`.
2. The total should not start at 1, it should be 0. Line 01 writes 1 to total. This line should be `Total ← 0`.
3. Only 4 numbers are input, it should be 5. There are two ways this can be changed, both are valid. Either Line 02 can be `Quantity ← 0` or line 03 can be `WHILE Quantity < 6`.

SKILLS FOCUS 7.8

FINDING ERRORS BY READING THE CODE

You may not always need to use a trace table, or you may not want to. There are many different ways of finding the errors. In this example you will look for each of the requirements in the algorithm to determine whether it has been met.

The following algorithm should take 3 numbers as input, and output the largest value and the average value.

```
01 Total ← 0
02 Largest ← 999
03 FOR X ← 0 TO 2
04     INPUT Data
05     IF Data < 999 THEN
06         Large ← Data
07     ENDIF
08     Total ← Total + Data
09 NEXT X
10 OUTPUT Largest
11 Average ← Total / 3
12 OUTPUT Average
```

CONTINUED

Re-read the description: take 3 numbers as input, and output the largest value and the average value.

This can be split into three sections:

1 Take 3 numbers as input.
2 Output the largest value.
3 Output the average value.

The two outputs need to be calculated before being output, so they can be added to the requirements:

1 Take 3 numbers as input.
2 Calculate the largest value.
3 Output the largest value.
4 Calculate the average value.
5 Output the average value.

Now check the first requirement:

Does it take 3 numbers as input?

The for loop goes from 0 to 2, so it will run 0, 1, 2 = 3 times. Inside the loop it inputs a value each time. **This is correct**.

The second requirement:

Calculate the largest value.

To find the largest it needs to:

a Initialise a largest variable with a low number.

This is **not** met. Line 02 largest is set to 999 but this should be a very small value, e.g.

```
Largest ← -1
```

b It should check if the data input is larger than the data in largest.

This is **not** met. Line 05 checks if data is less than 999. It should be larger.

```
IF Data > Largest THEN
```

c Line 06 replaces the value in `Large` with data, but the identifier is **incorrect**, the variable is named largest.

```
Largest ← Data
```

The third requirement:

Output the largest value.

Line 10 outputs the largest value, **this is correct.**

CONTINUED

The fourth requirement:

Calculate the average value.

The average is calculated by totalling the inputs and then dividing by the number of inputs.

Line 08 totals the data entered, but it uses the **wrong identifier**, it should be data not DATA.

```
Total ← Total + Data
```

Line 11 works out the average **correctly**.

The fifth requirement:

Output the average value.

Line 12 outputs the average **correctly**.

In this case the errors and corrections have been identified along the way. You can also combine this with using test data to check both the original algorithm and to check that your corrections are accurate.

Questions

1 Complete the trace table for the following algorithm with the input data: 20 50 60 25 35.

```
Total ← 0
Average ← 0
Counter ← 0
WHILE Counter <= 5 DO
    INPUT Mark
    Total ← Total + Mark
    Counter ← Counter + 1
ENDWHILE
Average ← Total / Counter
OUTPUT "The total is ", Total
OUTPUT "The average is ", Average
```

Total	Average	Counter	Mark	OUTPUT

CONTINUED

2 **a** Complete the trace table for the following algorithm using the input data: 7.

```
INPUT Number
WHILE Number >= 0 DO
    OUTPUT Number
    Number ← Number - 1
ENDWHILE
```

Number	OUTPUT

b State the purpose of the algorithm.

3 Explain the purpose of the following algorithm:

```
Valid ← 0
Invalid ← 0
Continue ← "Y"
WHILE Continue = "Y" DO
    INPUT Value
    IF Value >= 10 AND Value <= 20 THEN
        Valid ← Valid + 1
    ELSE
        Invalid ← Invalid + 1
    ENDIF
    OUTPUT "Continue?"
    INPUT continue
ENDWHILE
OUTPUT "valid = ", Valid
OUTPUT "invalid = ", Invalid
```